

Validação *de dados*

Dado que chega de fora (formulário, planilha, API, `input()`) não é confiável até ser conferido. Validar é checar o **formato** (regex), o **tipo** (conversão com `try/except`), a **faixa** e as **regras de negócio**, e responder com mensagens claras. Nesta aula: e-mail e CEP, CPF com dígitos verificadores, números, datas reais, um formulário que junta todos os erros e como lançar seu próprio `ValueError` .

`fullmatch` para formato

CPF

`try/except ValueError`

`datetime.strptime`

Lista de erros

`raise ValueError`

Formação Claude Code 2026 · Baseado em docs.python.org (re; Errors and Exceptions; datetime.strptime; Built-in Exceptions: ValueError) e no algoritmo público dos dígitos verificadores do CPF, consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

E-mail e CEP *com fullmatch*

email_cep.py

```
import re

EMAIL = re.compile(r"[\w.+~]+@[~\w-]+(\.[~\w-]+)+")
CEP = re.compile(r"\d{5}-?\d{3}")

for e in ["ana@site.com", "bia@site",
         "caio @x.com", "d@x.com.br"]:
    print(f"{e:13} ->", bool(EMAIL.fullmatch(e)))
for c in ["01310-100", "01310100", "1310-100"]:
    print(f"{c:13} ->", bool(CEP.fullmatch(c)))
```

SAÍDA

```
ana@site.com  -> True
bia@site      -> False
caio @x.com   -> False
d@x.com.br    -> True
01310-100     -> True
01310100      -> True
1310-100      -> False
```

- ✓ Para validar, use `fullmatch`: o texto inteiro tem de seguir o padrão
- ✓ Com `search`, "lixo ana@site.com lixo" passaria
- ✓ `-?` deixa o hífen do CEP opcional

Regex de e-mail só confere o formato. Se o e-mail existe, só um e-mail de confirmação diz.

CPF: formato *e dígitos verificadores*

cpf.py

```
import re

def cpf_valido(cpf):
    numeros = re.sub(r"\D", "", cpf)      # só os dígitos
    repetido = numeros == numeros[0] * 11
    if len(numeros) != 11 or repetido:
        return False

    for tamanho in (9, 10):              # os 2 dígitos finais
        soma = sum(int(d) * (tamanho + 1 - i)
                    for i, d in enumerate(numeros[:tamanho]))
        digito = soma * 10 % 11 % 10
        if digito != int(numeros[tamanho]):
            return False
    return True

for cpf in ["529.982.247-25", "529.982.247-26",
            "111.111.111-11"]:
    print(cpf, cpf_valido(cpf))
```

SAÍDA

```
529.982.247-25 True
529.982.247-26 False
111.111.111-11 False
```

- ✓ `re.sub(r"\D", "", cpf)` aceita com ou sem pontos e traço
- ✓ Sequências repetidas (111.111...) têm dígitos "certos", mas não são CPFs válidos
- ✓ Os dois últimos dígitos são calculados a partir dos anteriores, com pesos decrescentes

Os CPFs são de exemplo, gerados só para testar a conta.
Nunca use CPF real em código de estudo.

Converter *e conferir os limites*

numeros.py

```
def ler_idade(texto):
    try:
        idade = int(texto)
    except ValueError:
        return None, f"{'texto'} não é um número inteiro"
    if not 0 <= idade <= 120:
        return None, f"{idade} está fora da faixa 0-120"
    return idade, "ok"

for entrada in ["34", "trinta", "-3", "150", " 42 "]:
    print(repr(entrada), "->", ler_idade(entrada))
```

SAÍDA

```
'34' -> (34, 'ok')
'trinta' -> (None, "'trinta' não é um número inteiro")
'-3' -> (None, '-3 está fora da faixa 0-120')
'150' -> (None, '150 está fora da faixa 0-120')
' 42 ' -> (42, 'ok')
```

- ✓ Tentar converter e tratar o `ValueError` é mais confiável que prever todos os formatos
- ✓ Depois do tipo, a faixa: idade negativa é número, mas não é idade
- ✓ `int(" 42 ")` ignora espaços nas pontas
- ✓ Devolver `(valor, mensagem)` deixa claro o que deu errado

A data *existe mesmo?*

`datas.py`

```
from datetime import datetime

def data_valida(texto):
    try:
        return datetime.strptime(texto, "%d/%m/%Y").date()
    except ValueError:
        return None

for d in ["05/10/2026", "31/02/2026",
         "29/02/2024", "2026-10-05"]:
    print(d, "->", data_valida(d))
```

SAÍDA

```
05/10/2026 -> 2026-10-05
31/02/2026 -> None
29/02/2024 -> 2024-02-29
2026-10-05 -> None
```

- ✓ Um regex aceitaria `31/02/2026` ; o `strptime` sabe que fevereiro não tem 31
- ✓ Também conhece ano bissexto: `29/02/2024` passa
- ✓ `"%d/%m/%Y"` é o formato esperado: outra ordem dá `ValueError`

Formato com regex, existência com a biblioteca certa: cada ferramenta no seu papel.

Mostre *todos os erros* de uma vez

formulario.py

```
import re

def validar(cadastro):
    erros = []
    if len(cadastro.get("nome", "").strip()) < 3:
        erros.append("nome: mínimo de 3 letras")
    if not re.fullmatch(r"[\w.+~]+@[~w-]+(\.[~w-]+)+",
                       cadastro.get("email", "")):
        erros.append("email: formato inválido")
    if not str(cadastro.get("idade", "")).isdigit():
        erros.append("idade: use só números")
    return erros

bom = {"nome": "Ana", "email": "ana@site.com", "idade": "30"}
ruim = {"nome": " B ", "email": "b@", "idade": "trinta"}
print(validar(bom))
for erro in validar(ruim):
    print("-", erro)
```

SAÍDA

```
[]
- nome: mínimo de 3 letras
- email: formato inválido
- idade: use só números
```

- ✓ Em vez de parar no primeiro problema, a função junta todos numa lista
- ✓ Lista vazia significa cadastro válido
- ✓ `.get(campo, "")` evita erro quando o campo nem veio
- ✓ `.strip()` impede que espaços contem como letras

Recusando valores *com ValueError*

levantar.py

```
def definir_desconto(percentual):  
    if not 0 <= percentual <= 50:  
        raise ValueError(f"desconto deve ficar entre 0 e 50, "  
                        f"veio {percentual}")  
    return percentual / 100  
  
print(definir_desconto(15))  
try:  
    definir_desconto(80)  
except ValueError as erro:  
    print("Erro:", erro)
```

SAÍDA

```
0.15  
Erro: desconto deve ficar entre 0 e 50, veio 80
```

- ✓ Quando uma função recebe um valor que não faz sentido, ela pode **recusar** com `raise`
- ✓ `ValueError` é a exceção para "tipo certo, valor errado"
- ✓ Quem chama decide o que fazer, com `try/except`
- ✓ A mensagem diz o limite e o valor recebido

Senha *forte*

Enunciado: diga o que falta para uma senha ser forte: 8 caracteres, uma maiúscula, uma minúscula e um número.

Uma resposta possível

```
import re

REGRAS = [
    (r".{8,}", "pelo menos 8 caracteres"),
    (r"[A-Z]", "uma letra maiúscula"),
    (r"[a-z]", "uma letra minúscula"),
    (r"\d", "um número"),
]

def problemas_da_senha(senha):
    return [msg for padrao, msg in REGRAS
            if not re.search(padrao, senha)]

for s in ["abc", "Senha2026", "senhafraca1"]:
    faltam = problemas_da_senha(s)
    print(s, "->", "forte" if not faltam else "falta:")
    for item in faltam:
        print("    -", item)
```

SAÍDA

```
abc -> falta:
    - pelo menos 8 caracteres
    - uma letra maiúscula
    - um número
Senha2026 -> forte
senhafraca1 -> falta:
    - uma letra maiúscula
```

- ✓ As regras ficam numa lista de dados: dá para acrescentar uma sem mexer na função
- ✓ Desafio: exija também um símbolo, como `!@#&$`

Teste o que *ficou*

1. Para validar o formato de um campo inteiro, use:

A) re.search

B) re.fullmatch

C) re.findall

D) re.sub

B.

2. int("doze") lança:

A) TypeError

B) ValueError

C) KeyError

D) Nada: devolve 0

B.

3. Qual pega "31/02/2026" como inválida?

A) r"\d{2}/\d{2}/\d{4}"

B) datetime.strptime

C) str.isdigit

D) len(texto) == 10

B.

4. Por que juntar os erros numa lista?

A) Para mostrar todos de uma vez

B) Para o código ficar mais rápido

C) Porque o raise exige

D) Para esconder os erros

A.

Confie, *mas confira*

Valide em camadas: o formato com `re.fullmatch`, o tipo convertendo com `try/except`, a faixa com comparações, a existência com a biblioteca certa (`strptime`) e as regras de negócio (dígitos do CPF) com uma função própria. Junte os erros numa lista para o usuário corrigir tudo de uma vez, e use `raise ValueError` quando uma função receber um valor sem sentido.

`fullmatch`

`try/except`

Faixa

`strptime`

Lista de erros

`raise`

Próxima aula → 08 · Usando Map

Fontes: docs.python.org/3/library/re.html; docs.python.org/3/tutorial/errors.html; docs.python.org/3/library/datetime.html#datetime.datetime.strptime; docs.python.org/3/library/exceptions.html#ValueError; algoritmo dos dígitos verificadores do CPF (módulo 11), consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.